

Lesson 10 — Enemies That Seem To Think

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

Decide, then act. Separately.

```
function updateEnemy(e, dt) {
  decide(e);          // should I change what
  I am doing?
  act(e, dt);         // do what I am doing
}
```

Hysteresis: two thresholds, one boundary

```
if (e.mode === "patrol" && d < SIGHT)
  e.mode = "chase";
else if (e.mode === "chase" && d >
  SIGHT+GAP) e.mode = "patrol";
```

Same number for both → the enemy flickers **every frame** at the edge. The band between the two is how much noise it tolerates.

Line of sight

```
const dir = to.sub(from).normalise();
const STEP = TILE / 2;           //
smaller than a tile!
for (let i = 1; i < dist / STEP; i++) {
  const p = from.add(dir.scale(i * STEP));
  if (isSolid(floor(p.x/TILE),
  floor(p.y/TILE)))
    return false;
}
return true;
```

Vision cone — one dot product

```
facing.dot(toPlayer) > 0.7      // ±45°
```

$\cos 30 = 0.87 \cdot \cos 45 = 0.71 \cdot \cos 90 = 0 \cdot -1$
= sees everywhere.

Steering

```
seek:  (target - me).normalise()
flee:  (me - target).normalise()

arrive:
  let speed = MAX;
  if (d < SLOW) speed = MAX * (d / SLOW);

wander:
  e.angle += (Math.random()-0.5) * 2.5 * dt;
// += NOT =

separation:
  for each other enemy within
  PERSONAL_SPACE:
    push += away.normalise()
    * (SPACE - d) / SPACE;
```

Separation is three lines and turns a stack into a pack.

Fair, not strong

	why
reaction time 200–400 ms	looks like realising
telegraph	the player can respond
inaccuracy	perfect aim reads as cheating
max turn rate	can be outmanoeuvred

The player must be able to see why they lost.

AI need not run every frame

Ten decisions a second is indistinguishable from sixty, and costs a sixth as much.

Section A — Recall

A1. Where does the illusion of intelligence come from, if not from any behaviour being clever? [2 marks]

A2. Why keep `decide` and `act` as separate functions? [2 marks]

A3. What is hysteresis, what bug does it prevent, and give one example from outside games. [3 marks]

A4. Why must the line-of-sight ray step in pieces smaller than a tile? What goes wrong otherwise? [3 marks]

A5. Why is a dot product better than comparing angles for a vision cone? [2 marks]

A6. What problem does *separation* solve, and why does it look like a bug in each individual enemy when it is missing? [3 marks]

A7. Name three deliberate handicaps that make an enemy better to play against, and say what each one buys. [3 marks]

Section B — Predict the output

B1. `SIGHT` is 180 and `GAP` is 70. At what distance does the guard start chasing? At what distance does it give up? How wide is the band where nothing changes? [3 marks]

B2. A player stands at exactly distance 180 with `GAP = 0`, and small movements take them a pixel either side. Describe what happens over one second at 60 fps. [4 marks]

B3. `TILE` is 32, the enemy is 200 px from the player, and `STEP` is 16. How many tile lookups does one line-of-sight test do? For 12 enemies at 60 fps, how many per second? [3 marks]

B4. `facing` is `(1, 0)`. The player is directly above the enemy, so `toPlayer` is `(0, -1)`. What is the dot product, and is the player inside a cone of `> 0.7`? [3 marks]

B5. `MAX_SPEED` is 200 and `SLOW_RADIUS` is 100. Give the arrive speed at distances 300, 100, 50 and 0. [4 marks]

Section C — Find and fix the bug

C1. The enemy changes state several times a second whenever the player stands still at a certain distance. [3 marks]

```
if (dist < SIGHT) { e.mode = "chase"; }  
else { e.mode = "patrol"; }
```

C2. Five chasers behave as if they were one enemy: they overlap exactly and move as a single sprite. Each one's code is correct. What is missing, and why does it look like an individual bug? [4 marks]

C3. The enemy twitches violently instead of meandering. [3 marks]

```
e.wanderAngle = (Math.random() - 0.5) * 2.5 * dt;  
e.vel = Vec2.fromAngle(e.wanderAngle).scale(SPEED);
```

C4. The enemy sees the player through a one-tile-thick wall, sometimes but not always. `TILE` is 32. [4 marks]

```
const STEP = 48;
```

C5. The enemy disappears permanently whenever it catches the player exactly. [3 marks]

```
const dir = player.pos.sub(e.pos).normalise();  
e.pos = e.pos.add(dir.scale(SPEED * dt));
```

Section D — Build it

Use your tilemap game from lessons 5 and 6.

- ☐ **Checkpoint 1.** Add one enemy with a `mode` and separate `decide` and `act` functions. Draw the mode on screen above its head — keep that label all lesson.
- ☐ **Checkpoint 2.** Patrol and chase, with *one* threshold. Stand at the edge and count the state changes on screen. Then add the gap and watch the counter stop.
- ☐ **Checkpoint 3.** Add line of sight. Draw the ray, and draw it in a different colour when it is blocked. Hide behind a wall and check you are invisible.
- ☐ **Checkpoint 4.** Add a vision cone with the dot product, and draw the cone. Sneak up from behind.
- ☐ **Checkpoint 5.** Add four more enemies, then add separation. Turn it off and on while they chase you. This is the moment the lesson pays.
- ☐ **Checkpoint 6.** Add *arrive* so they stop at you instead of orbiting, and *wander* so the patrolling ones do not look as if they are on rails.
- ☐ **Checkpoint 7.** Add the four fairness handicaps, each on a key: reaction delay, telegraph, inaccuracy, turn rate. Play with all four off, then all four on.
- ☐ **Checkpoint 8.** Make `decide()` run only ten times a second instead of sixty. Try to tell the difference by playing. Then put the cost on screen and compare.

DRAW EVERYTHING THE ENEMY KNOWS

The mode, the sight ray, the cone, the distance, and the target it is moving

towards. Enemy bugs are nearly impossible to reason about from the code and nearly trivial to see once the enemy's own information is on the screen.</p>

Section E — Design challenge

NO RIGHT ANSWER

Marked on reasoning, and on naming the cost.

E1. What is the smallest number of times per second an enemy could reconsider before a player noticed? How would you find out, and does the answer depend on what the enemy is doing?

E2. When a guard gives up a chase, should it return to the start of its route, carry on from where it is, or go to where it last saw you? Each makes a different game. Pick one and say what it implies about how the player should play.

E3. Design an enemy that is genuinely hard and entirely fair: the player should lose and know exactly why. List what it does, what it telegraphs, how long each telegraph lasts, and the player's counter to each.

E4. The hard one. A guard who notices you, loses you, then **searches**: goes to where you were, looks around, gives up after a while. The enemy has to remember something, and the memory must expire. Where does that live, and what does it do to your state machine?

Stretch goals

1. Search (E4).
2. Alert nearby enemies when one spots you — one distance test, whole new game.
3. A flyer whose wander is biased towards the player, so it drifts in on a curve.

4. One difficulty dial from 0 to 1 driving reaction, accuracy, speed and turn rate.

Then say which of those actually changes the difficulty.

5. Three chasers that approach from different sides.